# Scoring and Write-up Guidelines

Laura
Updated 27 days ago

Follow

The purpose of the write-up is to report on the performance of the candidate based on what occurred during the interview.  While clients will have the ability to watch videos of each interview, our goal is to provide them with an accurate, detailed summary of what happened in the interview so that they only need to watch the video in rare cases.

This guide is intended to provide guidance and helpful examples to enable you to write high quality reports and to be precise and correct in scoring an interview.

After an interview, Interview Engineers complete a result form. This result form includes freeform text as well as checkboxes and radio buttons. The checkbox and radio input selections contribute to a candidate's score.

Detailed explanations will be given for how to evaluate candidates against assorted scoring metrics and through written thoughts sections.

This guide is loosely broken down by portion of the candidate assessment form, and concludes with some general Karat stylistic writing guidelines.

## Scores and Recommendations

The score is used to place candidates in one of four **Karat recommendation categories**:

- Fast track to next round
- Invite to next round
- Requires further review
- Do not pursue

The scoring thresholds for each bucket are specific and carefully adjusted for each client.

Clients use the score and Karat Recommendation as factors in the decision to advance or decline the candidate within their hiring process.

Different clients will have different policies with regards to scores. For example, some clients may read all write ups to look for a diamond in the rough, and some may ignore all candidates in "Requires further review".

Because scoring has such an **impact on whether someone gets hired**, it is vital that Karat provides **consistent results**. This consistency is also what sets Karat apart from many internal interviewing teams.

For any answer that is at your discretion (such as an extra credit), if the discretionary credit is given, you must leave a QC note and request a QC review.

## Working with Quality Control

The Quality Control (QC) Team is composed of Interview Engineers that have extensive experience conducting and reviewing interviews.

The QC Team works to ensure that interviews and write-ups meet Karat Standards and client expectations.

Once you begin to conduct client interviews, interviews that you conduct will be reviewed by a member of the QC team.

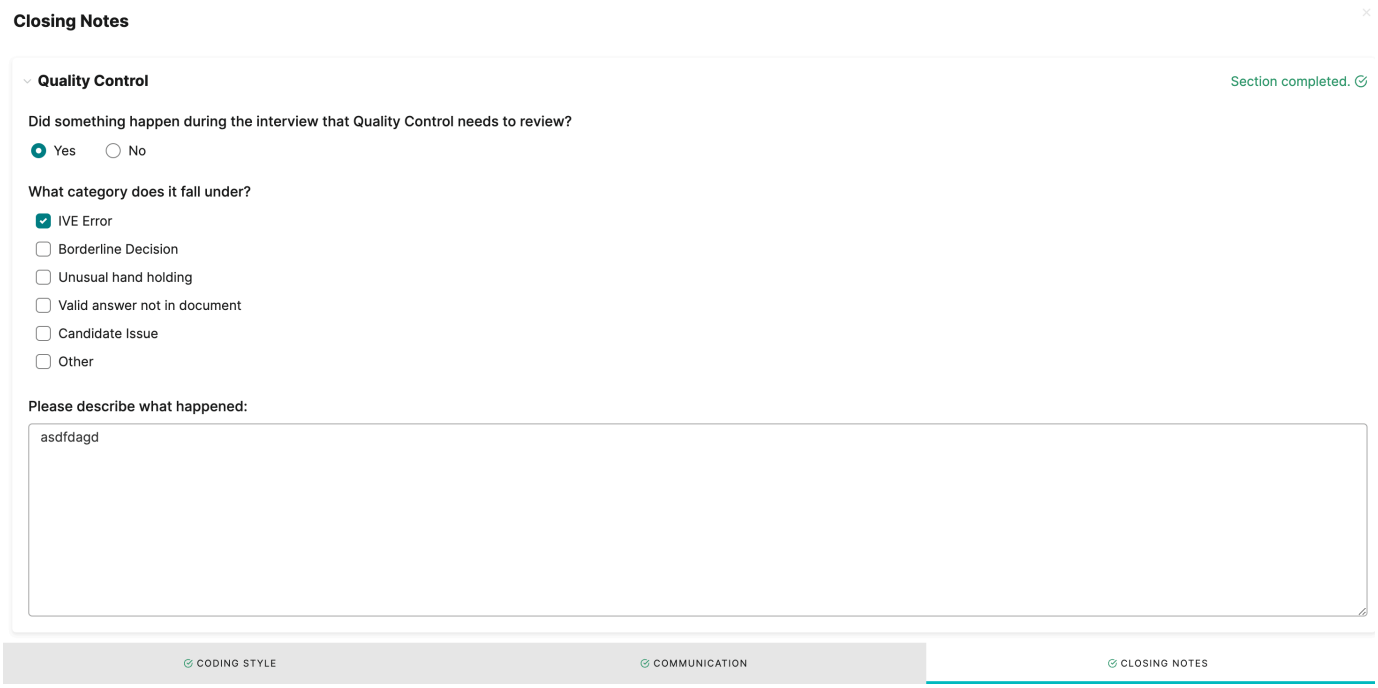
The QC Team member will

- Ensure that the write-up reflects what happened in the interview and meets Karat Standards
- Provide you with feedback on your adherence to Karat Guidelines, your conduct in the interview, your scoring of the interview, and the quality of the write-up
- Complete a QC Form (which will include the feedback)

### QC Notes

There are two sections to leave QC Notes. One section is for when there is something specific in the interview that requires Quality Control to review, and a second section is for additional notes/context that might be useful for a QCer to know, but don't require specific action.

"*Did something happen during the interview that Quality Control needs to review?*"




Answer “Yes'' if you believe there is something specific that QC needs to review. QC Reviewers will be reviewing this data to ensure that IVEs are leaving notes in the correct sections. After answering “Yes”, two additional questions will appear. Please review the definitions of the categories below to ensure you choose the correct one.

- **What category does it fall under**?
  - IVE Error: A mistake was made during the interview which negatively affected the candidate, such as forgetting to ask for the complexities, incorrect section timing, etc.
  - Borderline Decision: There are two types of info that could be included in this section
    - Some solutions fall right on the boundary between Trivial/Partial and Partial/Mostly. If a solution appears to be Trivial or Mostly Working but is being scored as Partial, this must be justified so it can be reviewed.
    - Scoring Uncertainty/Second Look: Occasionally a candidate will provide a novel answer to a DAQ or write a complex solution or use an obscure coding language. In these cases, to ensure the accuracy of the scoring, if unsure, a QC Note should be left for a second opinion from the QCer. This note must explain the reasoning behind the scoring originally selected.

  - Unusual HH: If Handholding was given to the candidate in an instance typically not supported in the guidelines, such as Moderate+ HH in Q2+, this must be explained so it can be reviewed by QC.
  - Valid answer not in doc: If a candidate gives an unlisted answer that you feel deserves credit, a note should be left so a QCer can review that answer. Also, a Content Request should be filled out so the Content Team can officially evaluate the answer and potentially update the guide.
  - Candidate Issue: If the candidate leaves the interview early or has technical issues that directly impacted the candidate's performance. For technical issues that just required pausing the timer, please do not use this field. Instead answer "No" and leave the information in the "Anything you'd like to note?" section.
  - Other:  For any case that does not fit in the above, but requires action by QC. This **does not** include if there are issues in two or more categories, where the most critical of the above should be selected.

- **Please describe what happened**:
  - Use this space to provide detailed information on what happened.

Answer “No” if there isn’t something specific that needs to be reviewed by QC. You can use this notes section to add context that will help the QC Reviewer be aware of tricky parts of the interview or report so that they don't have to guess or search. For example:

- It is a very helpful practice to leave general timestamps that the QCer may want to review.
- Always leave a brief note in case of technical difficulties, if any of the interview sections was extended for any reason, or if the timer was paused for more than a few seconds total.
- An error was made that **did not** directly impact the candidate or scoring, such as forgetting a piece of a section intro, or forgetting a custom script note.

***When you answer “Yes”, and trigger a QC to be completed, the issue you flag in the accompanying note will be waived from a Take Note (TN). However, excessive reliance on this exemption or aggregious issues may still result in a TN being issued.*

### QC Appeals

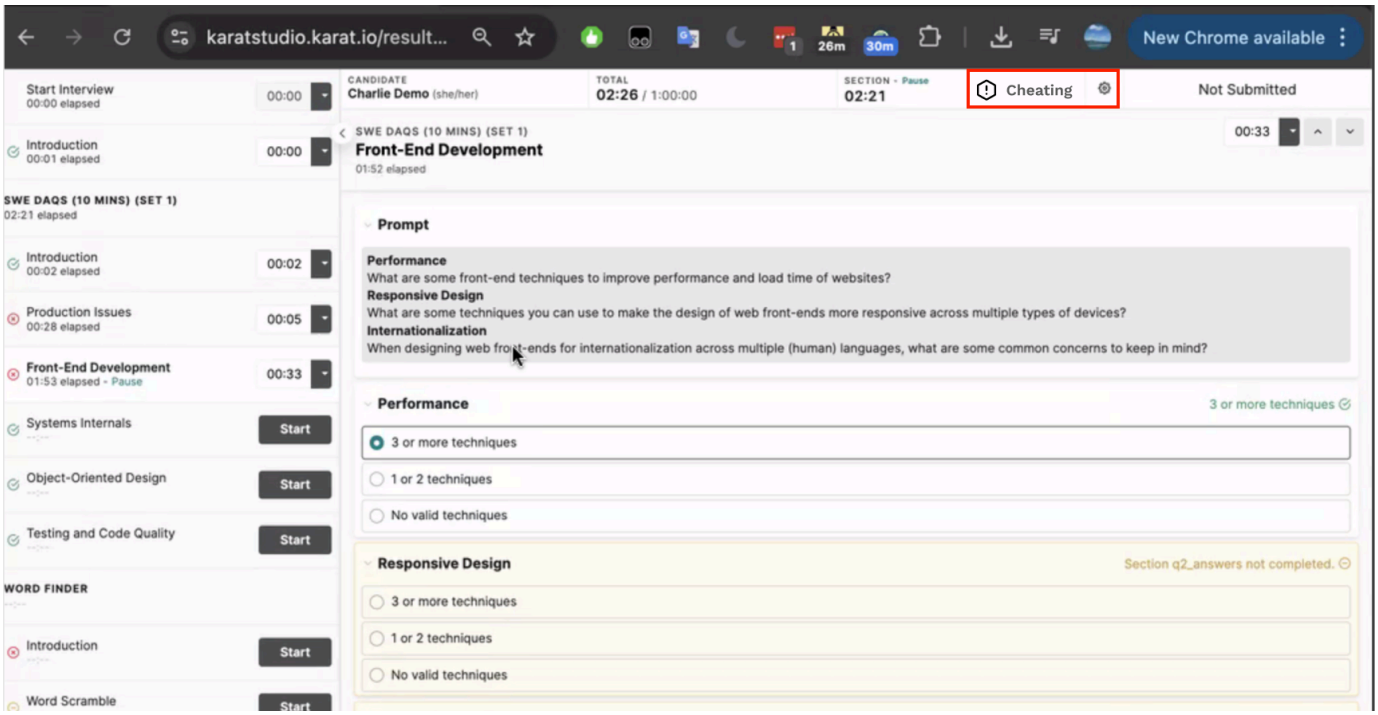
- A QC Appeal is filled out when an IVE disagrees with a QCer and a consensus has not been reached in the IVE QC category in Vanilla
- We have taken some steps to speed up the QC Appeal process, and newly received appeals (which are where TNs are most relevant to performance metrics) are being processed much more quickly than previously.
- Mentors **are not permitted** to override a TN without agreement between the IVE and the QCer and they should not be pinged into these discussions. The official QC Appeal process must be followed, using this form.

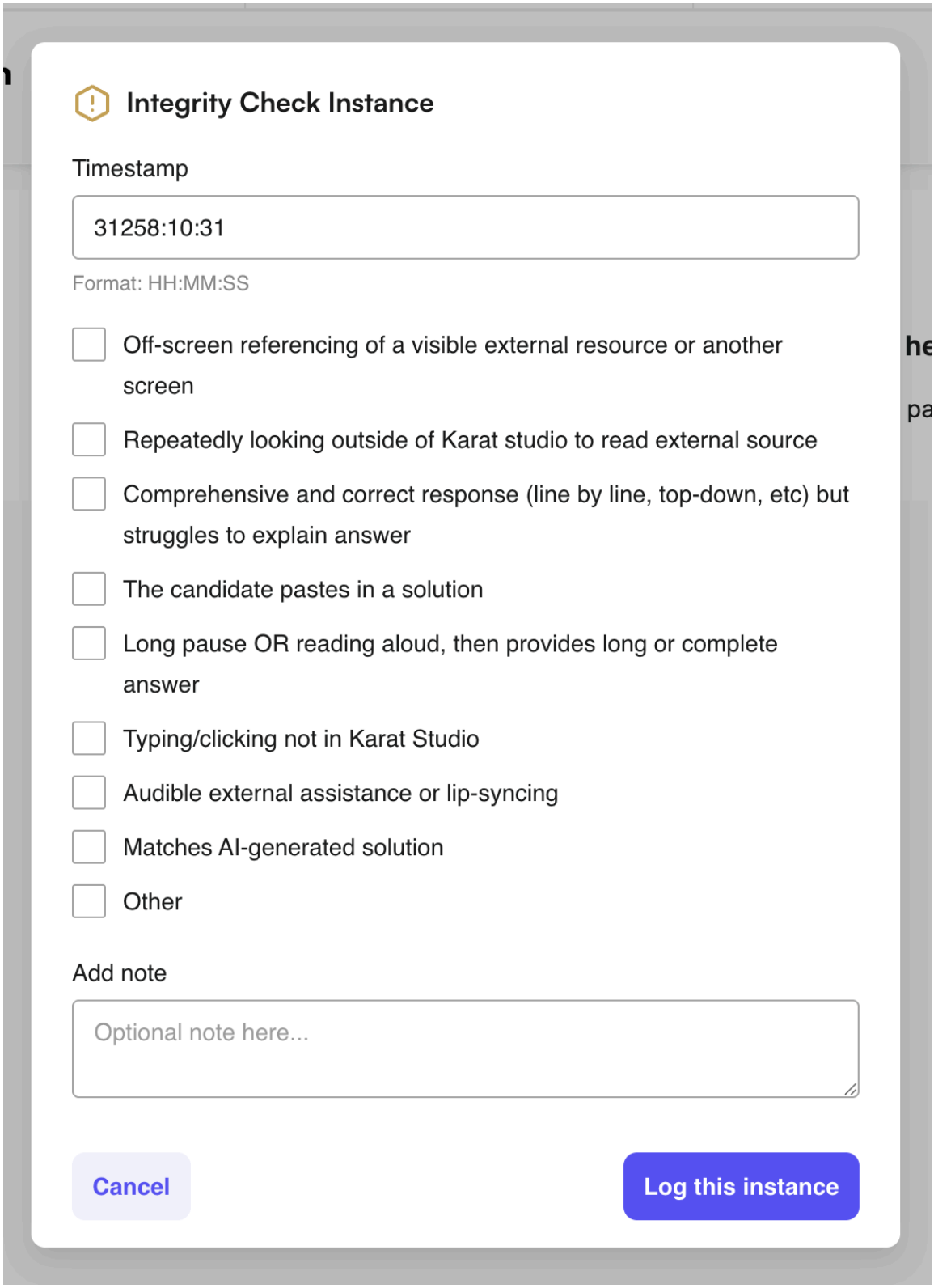
All of this helps us improve the consistency of our QC program and the overall quality of our interviews.

### All integrity Checks

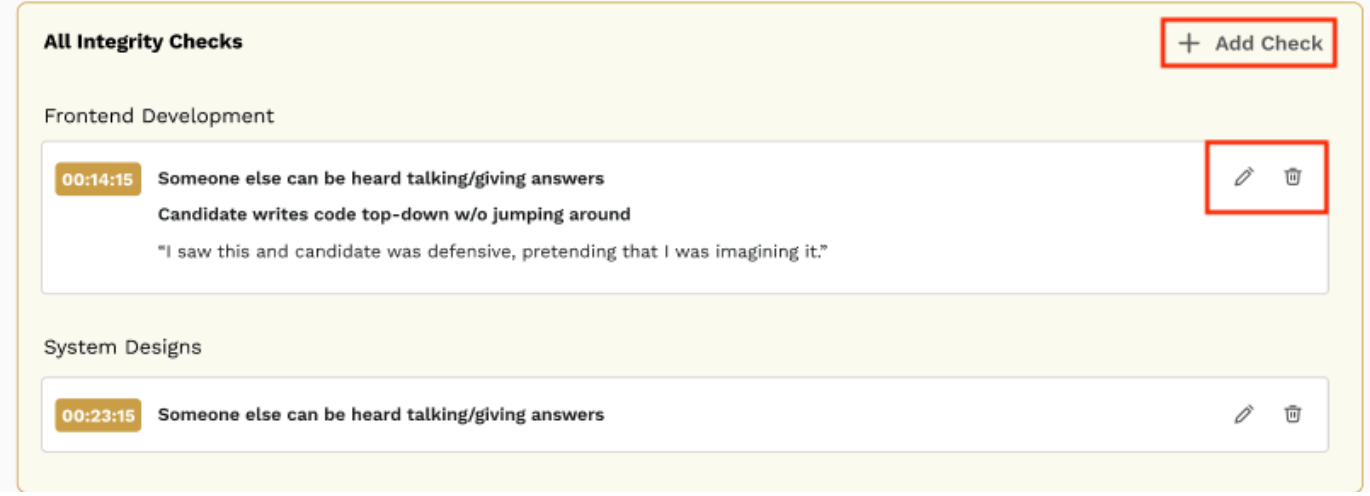
Use the Integrity Check feature in the results form to flag suspected cheating instances in real time. The feature can be used in two ways :

- Click the **Cheating button**, add or adjust the timestamp, and select the reason.


karatstudio.karat.io/result...
New Chrome available
Start Interview
00:00 elapsed
00:00
Introduction
00:01 elapsed
00:00
SWE DAQS (10 MINS) (SET 1)
02:21 elapsed
Introduction
00:02 elapsed
00:02
Production Issues
00:28 elapsed
00:05
Front-End Development
01:53 elapsed - Pause
00:33
Systems Internals
Start
Object-Oriented Design
Start
Testing and Code Quality
Start
WORD FINDER
Introduction
Start
Word Scramble
Start
CANDIDATE
Charlie Demo (she/her)
TOTAL
02:26 / 1:00:00
SECTION - Pause
02:21
Cheating
Not Submitted
SWE DAQS (10 MINS) (SET 1)
Front-End Development
01:52 elapsed
00:33
Prompt
Performance
What are some front-end techniques to improve performance and load time of websites?
Responsive Design
What are some techniques you can use to make the design of web front-ends more responsive across multiple types of devices?
Internationalization
When designing web front-ends for internationalization across multiple (human) languages, what are some common concerns to keep in mind?
Performance
3 or more techniques
3 or more techniques
1 or 2 techniques
No valid techniques
Responsive Design
Section q2_answers not completed.
3 or more techniques
1 or 2 techniques
No valid techniques




- If suspected cheating wasn't flagged during the interview, an integrity check can still be added afterward in the Closing Notes section. Click on the add check button on the top right corner of the section to add or adjust the timestamp, and select the reason. Multiple instances can be flagged if needed. QCers will review these flagged moments, confirm or edit them.




## Karat Writing Conventions and Common Mistakes

Interview write-ups should only say as much detail as necessary--just enough to justify the related scores chosen. **Be sure to always fill out the "Thoughts" section for each major section, except for the "Thoughts" area of coding questions that were not asked in the interview.** There is no need to be verbose, the goal is to simply state the facts about what happened during the interview so that clients can make a well informed hiring decision.

Karat Writing Conventions are general rules that have evolved over time to give consistency to the tone and style in writing that Karat clients have come to expect.

This is not an exhaustive or definitive ruleset, but rather a collection of best practices. Adhering to these rules will keep you clear of the most frequently corrected convention mistakes. In general, you should not **depart** from these rules.

### The Karat Voice

The Karat voice tries to be objective and non-judgmental, since every client has a different bar and values different competencies. We're presenting facts. By following these guidelines, we hope we can create more consistent profiles that accurately describe the candidate's performance to showcase to our customers. Moreover, we've included some frequently occurring issues that our editors correct.

### Avoid Opinions & Assumptions

- We focus on absolute performance, not relative performance. In other words, we do not say things like "That candidate coded slowly **for someone with 7 years of experience**". Instead you should just say that the candidate's coding fluency was proficient (if this is what you mean by "slow for having 7 years of experience"). Every company we work with sets their own hiring bar, which can vary dramatically. Our job is to rank each candidate against an objective standard (or as close to one as we can); the client can decide whether a candidate has outperformed for their experience level and whether that matters to them.
- Be careful stating opinions when there are assumptions built into the statement. For example, "She is clearly a skilled problem-solver that is able to mentally tackle complex programming concepts." Instead, it's better to say that she clearly solved these particular problems in your write-ups as we are not sure she is as clear a problem solver in other contexts.

- Candidates may choose the pronoun they wish to be identified by when they schedule their interview. We always respect a candidate's pronoun choice and reflect their choice in the writeup. Never make an assumption about a candidate's identity. If the candidate does not select a pronoun choice, use a combination of they/them and/or their NAME (not "the candidate") throughout the writeup.
  - If you feel like you need an accommodation in place of using a candidate's preferred pronoun in the write up, please notify your Relationship Manager. You will be allowed to use the candidate's name (not "candidate") in the writeup in place of the pronoun. We also recognize that not all candidates want to identify a pronoun; if a candidate did not select a pronoun at scheduling, the THEY/THEM pronoun macros in the writeup form will default to using the they/them in place of the pronoun.
  - Please refer to the samples with pronouns section for tips and examples on writing without pronouns.

## Grammar

Many Interview Engineers use a service called Grammarly that is installed as a browser extension. It can alert you of common grammatical mistakes and assist you in proofreading as you write.

Here are some general grammatical guidelines to follow:

- Write in third person. Use either "the interviewer" to refer to yourself, or reword to avoid adding yourself as a subject at all.
  - "I prompted him to stop looking after defining the rectangle." (Incorrect)
  - "The interviewer prompted him to stop looking after defining the rectangle." (Correct)
  - "He was prompted to stop looking after defining the rectangle." (Correct)

- Make sure to use the right indefinite articles
  - a -> an
  - Forgetting the word "the"
  - & -> and

- Verbs should be in past tense, in line with describing facts that happened, rather than generalizations about how strong a candidate is overall.
  - "is quick" -> "was quick"
  - "understand" -> "understood"

- Avoid incomplete sentences

## Guidelines

- **Do not use "was able", "was not able"** or **could (not) do** when describing what the candidate did or did not do. We don't speak about the abilities of the candidate in general, only about what the candidate actually did or did not do in the interview.
- When describing the approach of the candidate, **do not say that the candidate "came up with" an approach.** This wording tends to diminish the accomplishment of the candidate. You can alternatively use "formulated", "constructed", "identified", or any other formal wording to describe the process of working out the approach or choosing test cases.
- All numbers less than 10 should be written out as words instead of numerals, eg. one, two, three instead of 1, 2, 3. Numbers greater than or equal to 10 can be written out as words or as numerals.

- Please use "the interviewer" to refer to yourself rather than speaking from the first person perspective with the use of "I" or "we."
- Refer to the candidate by first name rather than "the candidate."
- Please ensure that your writing contains complete sentences. Make sure that you include the subject in each sentence. Bullet points and numbered lists are fine as long as they are professional and easy to understand.
- Please make use of single backticks ( \` ) to surround the inclusion of specific types, method names, Big-O notation or other items that should be represented as highlighted monospace text. This is a feature of Markdown.
- Do not be concerned about your report being original or novel from one report to the next.
- When making reference to a specific dimension of scoring, like completeness, approach quality, or handholding, be sure to use the same terms that are used in the form.
    - For example, instead of writing it like this --
        - "Billy wrote a nearly complete solution for the first coding question."

    - Write it using the proper scoring terminology:
        - "Billy wrote a mostly working solution for the first coding question."

- Always use the past tense where possible.
    - This includes projects that are currently ongoing in the project discussion.

- Avoid repetition
    - E.g., you don't have to write down the approach if you've selected an approach
    - E.g., you don't have to state the project was of "Average Difficulty" if already marked. (Unless you're about to explain why you marked the  difficulty as average, and you need to mention it to set the context for your explanation.)

- When making reference to a particular coding question, refer to it as "the first coding question", "the first coding problem" or similar, as appropriate. Avoid "Question One", "question one" and alike.
- Avoid words such as "very", "really", "extremely", and so on. In most instances, these words can simply be deleted. They are vague descriptors that don't add to the write-up.
- Use formal language.
- Avoid incorrect capitalization and incorrect spelling of technologies -- eg write JavaScript, MongoDB, etc. -- if you're unsure, look it up!
- Avoid vague statements, and words such as "some", which is defined as "being of an unspecified amount or number".
- Avoid colloquialism
- Be careful about calling code ["idiomatic"](#idiomatic-code) if you aren't **certain** it is
- Keep in mind that the client and other people reviewing the candidate's profile may not have watched the video, and may only have a few minutes to review the profile. Set context for information that is not covered by the default form fields. A client should be able to get the full picture just from reading the scores you entered and the text you wrote. For example, if the candidate switched approaches mid-way through a coding problem, don't just say "Her second approach was better and resulted in a working solution." You don't need to be overly verbose, but add some context: "Her initial approach of using exclusively goto statements was not working well. She switched to using a nested for loop instead, which resulted in a working solution."

### Examples

| Bad | Good |
| --- | --- |
| His communication skills are good. (Generalizing) | He communicated well. |
| Bad English, but he understands the domain very well. (Incomplete sentence; generalizing.) | While difficult to understand at times due to his mastery of the English language, his communication was effective, and he displayed solid technical domain knowledge. |
| She is clearly a skilled problem-solver that is able to mentally tackle complex programming concepts. (Generalizing) | She easily solved the first three coding questions and discussed a potential solution to the fourth question |
| Her efficiency issues were fixed after a few hints from the interviewer. (Lacking context.) | Her implementation ended up revisiting every element unnecessarily. Once this was pointed out, she quickly fixed the problem |

### Idiomatic Code

Be careful about calling code that you observed in the interview "idiomatic" in your writing.

"Idiomatic" code refers to a specific, opinionated style that usually comes after working with focus and close integration with the community and ecosystem of a particular coding language.

Just because certain code is "clean" or "readable" does not make it idiomatic. If you call code idiomatic mistakenly, your credibility in assessing code in that language could be called into question by a hiring manager that is more aware and cognizant of what is classified as "idiomatic" in that language.

For more information, see this blog post.

## General Write-up Structure

### Standard Format

- Project Discussion
  - Level of Difficulty (radio)
  - Level of Understanding (radio)
  - Description (4+ sentences)
  - Evaluation (2-3 sentences)
- Coding Question
  - Approach (radio)
  - Completeness (radio)

    - Handholding (radio)
    - Debugging (radio)
    - Time and Space Complexity (radio)
    - Thoughts (2-4 sentences)

- Coding Style
    - Syntax (radio)
    - Style (radio)
    - Fluency (radio)
    - Thoughts (2-4 sentences)

- Communication
    - Clarity of Speech (radio)
    - Quality of Explanation (radio)
    - Comprehension (radio)
    - Interaction (radio)
    - Thoughts (2-4 sentences)

## Discussion and Analysis Question (DAQ) Format

Only the sections that differ are listed. Other sections are as outlined in the Standard Format.

- Discussion and Analysis Questions
    - Candidate Answers (for each section) (checkbox)
    - Thoughts (2-3 sentences. Ensure what the candidate said is accurately summarized.)

## Offline Coding Challenge Format

Only the sections that differ are listed. Other sections are as outlined in the Standard Format.

- Offline Coding Challenge
    - Level of Understanding (radio)
    - Time & Space Complexity (radio)
    - Thoughts (2-3 sentences)

## Note on content length of Thoughts

The estimated length recommendations are provided with the intent of helping you know the typical range of what is acceptable. Please do not omit important information from Thoughts sections if it should be there.

Interview Write-up Quality is an important aspect of the value we provide to our clients. When we leave out important details from the thoughts section, we harm the client's ability to gain insight into the suitability of the candidate.

This is something that is monitored and reviewed in Quality Control and Performance Management. If you are omitting important details and consistently submitting sparse reports by only holding to the minimum length, you may be notified that this is unacceptable.

# Project Discussion

## Project Discussion: Scoring

Scoring for project section is difficult and subjective. We rate the **Difficulty** of the project and the **Understanding**.

We expect that a strong candidate has worked on something difficult and they have a strong understanding of that project.

We rate Level of Difficulty and Level of Understanding independently. The available scores for each of these are:

- Minimal
- Moderate
- Above average
- Significant

We also have a "N/A Not enough info" option for if the project was not discussed, or if the candidate's explanation was extremely difficult to understand. This should be the case only exceptionally, and you should make it clear in the project description and QC notes why you picked this option.

This lesson provides a set of examples of project scores, and Interview Engineers are

expected to map each interview to the closest of these examples in terms of difficulty and understanding.

## Project Discussion: Level of Difficulty

Nothing in life is inherently easy or difficult. Tasks are intrinsically simple or complex, and difficulty is a product of experience and the complexity of the task. We are scoring based on how complex the project compared to other projects was and not how hard it was for the candidate. That is, creating a basic CRUD site is hard for a beginner, and creating an image recognition tool may be easy for an image recognition expert, but in our scoring system the latter is scored as harder than the former.

Projects can be difficult in multiple ways. For example:

- The scope of the project is large and requires a lot of domain knowledge.
- The project pushes some boundaries of what is possible with existing tools.
- The project is done to a rigorous standard with a high requirement for correctness or performance.

We are scoring mainly based on the technical difficulty of the project. That means that a simple to implement project cannot be marked as "Significant" difficulty if it involved dealing with team bureaucracy or unprofessional colleagues.

### Level of Difficulty Options

#### Minimal

A project with a minimal level of difficulty is basic. It could be completed by someone with a low level of technical experience or skill, typically by piecing together a few libraries or tools.

Examples include:

- Creating a basic CRUD website
- Doing a school project
- Playing a small part in a complex project
- Converting an AngularJS component to React
- Creating a website/game in a short amount of time at a coding bootcamp
- Using an existing machine learning model or library to categorize images

### Moderate

A project that presented some obstacles is scored as moderately difficult. Most projects out in industry fall into this category.

Examples include:

- Working on part of a team creating an API to an existing database
- Adding tests to an existing service
- Developing a machine learning pipeline that trains a model for use in a production system.

### Above Average

A challenging project that involves collaboration with multiple disciplines is of above average difficulty.

Examples include:

- Writing a thoroughly tested trading system which is very performant
- Implementing a permissions system for a new service
- Creating an ETL system that processes very large volumes of data
- Creating and deploying a novel machine learning model

### Significant

Significantly difficult projects solve a problem that most people could not solve. This is a rare choice and indicates a special candidate.

For example:

- Modifying a popular programming language in a significant way
- Handling an unusually large scale in terms of users or data
- Making a game with a technically novel mechanic

## Project Discussion: Level of Understanding

Understanding of a project can be demonstrated in multiple way. A candidate may show a deep understanding of the business requirements of a project, or the technical stack used, or the technical problem space.

A strong candidate will show an understanding of the user or client needs, the technical choices made and the impact that these had.

### Project Level of Understanding Options

#### Minimal (Project)

A candidate knows only the basics of what was implemented, if at all. They do not know where the project fits into the bigger picture. They do not have an understanding of the reasons for technical decisions or the impact of these decisions.

#### Moderate (Project)

The candidate can explain the project's use from end to end. They may not know all of the technical details of other components, especially if it was a team project. They may not have been involved in key decisions.

#### Above average (Project)

The candidate provided information about why certain technologies were used. This may include the programming language and infrastructure selections such as the database. They understand technical details of other components that interface with this project.

This candidate may also demonstrate domain expertise such as expertise in image recognition or distributed systems. They may also demonstrate business context knowledge such as a deep understanding of the users' needs.

#### Significant (Project)

The candidate understands where the project fits into the larger picture.

They show a deep understanding of the technical decisions and intricacies of the project. They understand the technical and business value that the project brings to the company. The candidate understands some client issues, operational issues, and the future roadmap of the project.

They understand the requirements of the project in dimensions such as correctness, performance and features.

#### N/A Not enough info

This would happen if the project was not discussed at all.

## Including links in the writeup

Some candidates provide a link to showcase their work. As these are outside of Karat control, they may pose a security risk, so only include them if you are **certain** they point to well-known sites such as gitlab.com or github.com. If in doubt, omit them.

## Project Discussion: Description

The Project description section should contain facts about the project as they were conveyed in the interview. It should mention aspects like:

- Use four to five sentences
- Items to include:
  - General Project information
    - Motivation for project
    - Goals
    - Role of candidate with respect to code
    - Time spent on project
    - Team size
    - Technologies involved
  - Technical challenge
    - Description
    - Resolution
    - Time spent
    - Technologies involved
    - Understanding

Note that you do not need to transcribe your entire conversation with the candidate. Clients can always look at the video if they want every detail. Instead, summarize the facts that are relevant to your evaluation.

### Project Discussion: Evaluation

The evaluation portion of this section should be your analysis of the project discussion. It should include an explanation of why the candidate received their scores in this section, which includes the project difficulty and the project understanding. You should talk about the level of difficulty of the project and if the candidate seemed to have a significant contribution and understanding about the project.

- Use at least two sentences
- Guiding questions
  - How well did candidate explain the project?
  - How much technical detail was provided?
  - How difficult was the project? Why?
  - Did the candidate have a good understanding of the project?

## Writing Samples (Interview Write-Ups)

**Note: our writing samples are provided as examples only. Your interview write-up must always be tailored to the specific candidate and interview experience. Focus on clear, objective observations that accurately reflect the candidate's communication, problem-solving approach, and overall interview performance.**

### Project Discussion: Writing Samples

#### Writing Sample 1

### Description

*Jean-Luc described a distributed cache system used for holding serialized objects. He was one of the architects on the team of 20 that worked for 18 months, and was primarily responsible for the notification and data synchronization systems. One key challenge they faced was handling nodes that were offline for extended periods of time and required significant amounts of data to re-synchronize. In the end, they used a manual re-synchronization process to avoid swamping the network with data transmissions.*

### Evaluation

*Jean-Luc's project had to address a number of key challenges in distributed systems, including resilience in the face of unreliable hardware. He gave a good description of the entire system, though occasionally got stuck in describing technology instead of answering the questions as asked. He realized that the manual approach to failed nodes was not ideal, but it was not clear his proposed alternative of FTP-ing files around would have addressed the network swamping issues.*

## Writing Sample 2

### Description

*Betty worked on improving the scalability of an SMS gateway written in .NET and C++. When she started on the project, the system could handle a maximum throughput of about 70,000 messages per day. Over a one-year period, this was improved to about ten million messages per day. She made several improvements to the existing code base, ranging from fixing improper exception handling to batching database insertions to replacing AppFabric Caching with Redis. Betty worked on a team of about 20 where she was usually tasked with performance improvement.*

### Evaluation

*This project presented many challenges, which Betty explained well at a high level and gave some detailed examples of her solutions. She also stated that her work resulted in her being granted two promotions within one year.*

# Programming Problems: Scoring

The strongest candidates

- Write fully working solutions
- Require no help from the Interview Engineer
- Choose approaches which are optimal in terms of time and space complexities
- Know the time and space complexities of their approaches

Clients also want to know if the candidate thoroughly planned their approach.

# Programming Problems: Optimality of Approach

The approach explains how the candidate solved the problem. Most approaches to a problem have already been identified with their corresponding optimality in the question guide.

Each question guide includes multiple approaches for each question (ex. Q1, Q2, Q3). Each of these have an associated time and space complexities. The approaches are ranked between:

- **No viable approach**
- **Naive**
- **Acceptable**
- **Optimal**

Match the candidate's approach with an approach in the question guide. **Both the textual description and the complexity of the implementation should match.**

Note: Some questions have optimality cut-offs or ranges. Eg: Anything above a certain time/space complexity is considered optimal, anything below a certain complexity is considered naive, and anything between two complexities is considered acceptable. Choose the correct bucket for these ranges if this is the case.

On rare occasions the candidate's approach will not match any of the approaches in the guide. In these cases, compute the time and space complexities of the solution and compare it against all solutions in the question guide. If the complexities are no worse than the worst optimal, score optimal. If not worse than the worst acceptable, it's acceptable. Any solutions with lower complexities are naive/basic. Then describe the new approach in the result form after choosing "Other" and choose the assessed optimality. Note that the optimality is downgraded if *either* the time or space complexity is worse. For example, if the worst listed Optimal solution has O(n) time and space, and the worst listed Acceptable solution has O(n^2) time and O(n) space, a solution with O(n*log(n)) time and O(n) space would be Acceptable. A solution with O(n^2) time and O(n^2) space would be Naive/Basic.

If a candidate describes multiple approaches, choose the one which they made the most progress on coding. Sometimes a candidate will describe an approach before coding and then implement a different approach. Ask the candidate to describe their new approach, and then use the implemented approach for the ranking.

A candidate's code can also be considered part of the description of an approach, although it is almost always in their interests to describe their approach verbally in case any key elements are missing and you cannot be sure of the viability of the approach.

If a candidate's approach description lacks detail that affects its optimality, but not its viability, you should assume the worst reasonable approach. For example, if their approach requires them to produce the intersection of two collections, you should assume they will use nested loops rather than sets and set intersection, unless the variables have already been declared as sets.

If the candidate did not at any point describe an approach which may have worked, and the approach is not clearly viable from the code, then regardless of implementation progress choose **No viable solution**.

## Note about the 'No viable solution' selection

Select this option only when the candidate did not at any point describe or implement anything that was on the right track to solve the problem.

If the candidate did not describe the approach in full, determine if the elements missing from their description could be written off as implementation details or edge cases that require relatively minor fixes. If so, the approach is probably a viable one.

Every questions has its own quirks, so make sure to check the pitfall section of the questions guides for characteristics of a non-viable approach. If you are unsure on whether a "missing element" is an **implementation detail** or an **element of a viable approach** and the guide is not clear, please leave a QC note explaining your rationale and file a content form requesting an explanation for that case. A good rule of thumb is, if the problem will appear in testing and be easily fixable, then it likely is an implementation details. For example, a solution to Log Analytics Q2 which is missing sorting, or a DFS implementation missing a visited collection.

Note that the viability of the candidate's approach is independent of whether they successfully implemented any of it.

### Approach Checkboxes:

- Thoroughly planned and examined approaches
- Inadequate planning of approach
- Asked clarifying questions
    - Asking clarifying questions to make sure the problem is correctly understood is a positive signal

## Programming Problems: Completeness

A candidate's **approach** is an articulated plan or description of an algorithm.

A candidate's **solution** is the implementation of an approach.

Completeness is a measure of how correct the candidate's solution was. This typically is the state at the end of the interview, though it should be considered from the point where the candidate made the most progress during the exercise. If the state chosen is not from the end of the interview, it must be explained what happened in Coding Thoughts, usually this is when candidates delete working code when debugging.

Apart from with **No implementation** and **Fully working**, this leaves room for interpretation, but we try to be as consistent as possible.

When scoring borderline trivial or mostly-working solutions, please leave a QC note explaining the reasoning behind your decision. In these cases, it is important to have someone from the QC team to have a second look at it. Failing to do so may result in a Take Note.

### What if the candidate discussed a viable approach but went to implement a non-viable one?

In that case, select the viable approach they stated and score the code as if that was progressing towards that solution. That may mean their complete code solving the wrong problem could be for example optimal and have trivial (or partial) implementation, rather than non-viable approach.

### Completeness Options:

### No implementation

The candidate identified a reasonable approach, but ran out of time before writing any meaningful code. Writing only pseudocode/notes is included in this. If the candidate wrote syntactically valid code in comments (even if in a different language than selected), this still counts towards completeness.

## Trivial Implementation

The candidate wrote some code, but it did not solve any of the core problem. Includes:

- Re-using code from a previous question's solution without adding anything meaningful towards solving the current problem.
- Creating function stubs and initialising a few variables, but no actual logic.
- Sorting the input with no additional logic.

This is an example of code still considered Trivial, as the loops perform no meaningful work. This is considered the upper bound of Trivial, so more useful lines towards a solution would more the code to Partial:

```java
  private List<String[]> generateContiguousHistories(String[] user) {
   List<String[]> result = new ArrayList<String[]>();
   for (int j = 0; j < user.length)
     for (int i = 0; i < user.length - j; i++) {
       result.add(user[i]);
     }
   }
 }
```

## General Guidance

- If the candidate ran out of time after giving a reasonable approach, select "No Implementation."
- If they wrote some useful code, select "Partial."

## Partial

A partial solution is between a "Trivial implementation" and a "Mostly working" solution.

Examples of partial solutions include code which makes meaningful progress, but either no test cases pass or only a few trivial test cases pass. They may be passing one or two test cases, but be hard-coded to do so.

It is obvious that significant work remains to arrive at a fully working solution.

## Mostly working

There are two definitions of a mostly working solution.

If the solution fits either one, it is considered to be mostly working.

1. The candidate is clearly within five minutes of a fully-working solution, without moving the handholding level up.
2. The solution works in the general sense even if it doesn't run, however, it may not cover an edge case or two. When it works, it is not a result of hard-coding.

**Note**: Some candidates will continue to work on the problem after the interview. Any performance in the time after the interview cannot influence the scoring of the solution when time ended. This includes a solution that was Partial but the candidate actually did finish it within 5-minutes.

## Fully working

The solution is considered fully-working if it satisfies the following:

- The solution is written in a function and returns the output.
- The function works completely without any bugs, and all the test cases pass (including producing the expected behaviour for any additional edge cases outside the provided ones).
- The function returns the same result when run multiple times on the same input and return the correct result when running multiple sequential tests (say, due to global variable use or similar), the Interview Engineer should also point that out and ask them to fix it (please consult the handholding page). That's not to say they can't use global variables for storing data, but their function should ensure that they are reset between runs so that the function returns the correct result when run multiple times.

Note that the return data can be in any form the candidate chooses. As long as the output is correct, consistent, unambiguous, and works for all potential valid inputs, it is acceptable. It should be in the same rough form as our expected output; for instance if the expected output maps pairs of students to lists of shared courses, then the candidate's output should have some data structure that directly relates those two together such as a map or a list of `(student, student, courses)` tuples. It would not be acceptable to return a tree structure mapping `student -> (map of student -> list)`.

If the return type is ugly or difficult to work with, you may also address that in coding style section.

The candidate could also use out parameters, update pass-by-reference parameters, create a custom class with the data in it, or return some kind of generator function, but if this is non-idiomatic for the language you may want to call this out in the coding style section. This does not include storing the data in a global variable instead of returning.

Some candidates in Object-Oriented languages may chose to create a custom class to represent the whole solution. This could involve using the constructor or a function to generate the data and storing the data in fields on the class, or taking part of the input as a constructor parameter and the rest in a method. This is ok, as long as it satisfies the other requirements (it is especially worth checking that the solution will run correctly when called on the same input multiple times or with different inputs).

If the candidate's code passes the test cases but relies on some undefined behaviour (such as out-of-bounds memory accesses) and you can't break it during or after the interview, score it as fully-working but note it in the code style section.

## Additional requirements

The Interview Engineer should not move on to the next question without the code meeting the following additional requirements. However, if the Interview Engineer fails to enforce any of these additional requirements before moving on to the next question, then the candidate's code would still be considered fully working. The Interview Engineer should expect a Guidelines Take Note from QC though.

If the candidate does not have time to do all of these at the end of the interview, their code would still be considered fully working and this would *not* result in a Guidelines Take Note for the Interview Engineer.

- The function accepts the input in a format provided.
- If the candidate's code modifies the input then you should tell them to make a copy of the input.

### Completeness Checkboxes

- Did not account for edge cases
  - Note: This is meant to be selected primarily with partial or mostly working completeness, as it's intended to differentiate solutions because edge cases weren't handled vs. the implementation nearing completion.

## Handholding

The Handholding score is determined by the amount of guidance and hints given by the Interview Engineer to the candidate.

The Handholding score has four options:

- None
- Minimal
- Moderate
- Heavy

The Handholding score is determined by all of the handholding given, amalgamated.

The handholding score impacts the candidate's overall score. That is, the more handholding given, the worse the candidate's overall score is.

See the "Handholding" guide for what is considered handholding and how to choose when to handhold.

### What counts as handholding

When the candidate does not use the suggestions provided by the Interview Engineer, it is considered handholding in the write up.

Consider this akin to the amount of time spent holding someone's hand as they cross a tight rope.

If you provide handholding that the candidate did not need or did not use in their solution, it still counts as handholding and should be scored appropriately.

If you provide **incorrect** hints or advice, do **not** score it as handholding. This would be considered an interviewer error and candidates should not be penalized for such reason.

### Handholding Options:

See the handholding guide, which contains explanations of each level.

**None**

**Minimal (Handholding)**

**Moderate (Handholding)**

**Heavy (Handholding)**

### Handholding Checkboxes

- Helped formulate approach
- Helped with syntax errors
- Helped identify errors in approach
- Helped identify bugs or edge cases
- Helped fix bugs

### Adding up handholding

There is some natural grey area here. For example, if a candidate writes their own implementation but has 20 bugs and needs help fixing every one, at that point it's probably not just "major help finding bugs", it's "major help with solution", and therefore it should be scored as Heavy.

It is common for an Interview Engineer to choose a score of "Moderate", for example, after giving multiple "Minimal" guidances.

When in doubt, put whatever you think is the closest match to the criteria above; you can always ask for input from QC in the IVE QC category in Vanilla or leave comments about your concern in the **QC Notes** as well.

The "Thoughts" section is a great place to describe what the candidate failed to do without help. This can justify any score and helps someone doing Quality Control (QC) to help you to get more in line with Karat scoring standards. Choose the closest match and **ask for feedback in the notes to QC** if you are not sure.

## Programming Problems: Debugging

This field is intended to denote the amount of debugging that the candidate had to do on the problem. The idea is to differentiate between candidates who write fully working code immediately vs. those who need to repeatedly address problems with correctness. The bar for Minimal debugging should be anything beyond two or three compiler-type issues. Any logical mistakes constitute debugging. Heavy debugging should denote the candidate spending most of the interview debugging rather than writing code.

### Debugging Options:

**None (Debugging)**: The candidate's code worked on the first go or they only had to fix some small syntax errors / typos.

Minimal (Debugging): The candidate had to track down and fix one or maybe two small logical errors.

Moderate (Debugging): The candidate had to spend a few minutes on a larger bug or two

**Heavy (Debugging)**: The candidate spent a significant amount of time debugging a solution that wasn't working as intended

### Debugging Checkboxes

- Struggled with syntax errors
- Struggled with logic errors
- Off-by-one errors

## Programming Problems: Time and Space Complexities

For each coding question that is completed, the candidate should be asked what the time and space complexities are of their solution. If available direct the candidate to do this in the terms defined at the bottom of the question prompt.

If you are not sure what the time and space complexities of their solution are, **call this out in the notes to QC** on an interview.

The available options are as follows:

- If the candidate gives an incorrect time or space complexity, move on.
- If the candidate gives a time or space complexity which is ambiguous, such as `O(n)` for a function with multiple inputs, ask the candidate to clarify what `n` is. Having to ask for this clarification does not impact the score.

A candidate's response must be clear but does not need to be in perfect Big O syntax. For example, a complexity of `O(2n)` should be described as `O(n)` but we consider this correct.

Similarly, if a candidate describes complexity as "constant in terms of the size of the input", this should be considered the same as `O(1)`.

### Complexity Options:

**Correctly identified both**: The candidate correctly identified both the time and space complexities.

**Only identified time**: The candidate correctly identified time complexity but incorrectly identified space complexity.

**Only identified space**: The candidate correctly identified space complexity but incorrectly identified time complexity.

**Incorrectly identified both**: The candidate incorrectly identified both the time and space complexities.

**Was not asked**: The Interview Engineer did not ask the candidate to identify time and space complexities.

In the event that only time complexity was asked, a correct response should mark "Correctly identified both" and an incorrect response should mark "Incorrectly identified both".

If time or space complexities were asked, they should be scored accordingly, despite IVE's errors. In other words, if a solution is found to be mostly-working after time and space complexities were discussed, the candidate should still be given credit for the discussion.

## Programming Questions: Thoughts

The main scores for the entire interview are generated by the score labels for each question. The Thoughts section for each question should have a few sentences describing in detail how the candidate performed. **It should align with the scoring labels.**

For example, if handholding was scored as "Heavy" then the thoughts section should mention what hints were given that merited a score of "Heavy."

If there were bugs in the code (fixed or not), they should be mentioned here as well. Thoughts should be written up for each part of the technical question reached.

## Programming Question Overview

Use the text input to summarize the candidate's progress through each coding question.

Required items to include:

- A short explanation of what the help was if handholding other than none was given
- Time and/or space complexity answer if the candidate answered incorrectly
- If the score was mostly-working, an explanation of exactly what elements were missing from fully-working or of the test cases that were still failing.
- A short explanation (a sentence or two) of what the debugging was to justify the debugging score if other than none occurred.

Other items to possibly include if relevant:

- Optimality of the approach
- Level of completeness
- Any struggles in forming the approach or in the implementation
- Further detail for the time and space complexity analysis provided by the candidate
- When providing handholding (especially above minimal), it is appreciated to add a brief QC note with a rough timestamp as it speeds up the review
- If the candidate produces a fully-working optimal solution with no handholding or debugging, then talk a little about their planning or coding process to explain what contributed to their good progress. For example, maybe they planned in depth, or quickly turned their plan into clear code.

### Writing Sample 1 -- (Fully working, minor debugging, no handholding)

*Once again Frances instantly identified an optimal approach. She quickly recognized that the problem would require some type of BFS and steady implemented her approach, only pausing to deal with some minor logic errors around the checking if a pixel was previously visited. She finished up with five minutes remaining.*

### Writing Sample 2 -- (Fully working with heavy handholding)

*Simon knew several of the relevant PHP functions but struggled to correctly use them to address the larger problem. Among other issues, he did not seem to grasp the distinction between strings and arrays. The interviewer provided increasingly heavy guidance in helping him write code and resolve syntax errors. He developed a fully working solution in a bit over 40 minutes. He was confused about the distinctions between his variables. He incorrectly identified the space complexity to be `O(n*m^2)`, instead of `O(n*m)`, where `n` was the length of the input array, and `m` the average entry length.*

### Writing Sample 3 -- (Mostly working)

*Wilson quickly identified an acceptable approach and developed all parts of his algorithm but ran out of time before addressing a small syntax error around the usage of the map. He produced with no assistance a mostly-working solution that only missed an extra check for handling the case of 1x1 images.*

### Writing Sample 4 -- (Fully working with no handholding, debugging, or checkboxes selected)

*Shaun quickly identified an optimal approach and described it well. He developed a near perfect Python solution that worked correctly on the first run.*

## Code Style

### Code Style: Scoring

The Code Style scoring provides an indication of how well the candidate coded, as much as possible ignoring their method of finding valid approaches to the coding problems.

Scores should reflect the overall coding ability, and are often based on the Interview Engineer's judgment. If a candidate's coding proficiency appears to differ depending on the part of the problem at hand, choose the closest option to what you consider the average and optionally describe the situation in the "Thoughts" section.

There are three key metrics that we consider for Code Style:

- Syntax
- Style
- Fluency

For each of these there are checkboxes. These checkboxes aim to give a bit of color to the hard score. They are often derived from common "Thoughts" in write ups.

### Code Style: Syntax

The Syntax score is designed to show the client a candidate's proficiency and apparent familiarity with the programming language they chose.

What helps us know this is the following:

- Can they implement simple code structures without help or mistakes?

- Can they implement more complex code structures without help or mistakes?
- Is their code idiomatic?

This does not cover typing errors.

An Interview Engineer who is familiar with a language is more likely to know whether code is idiomatic. However, in many languages it is quite clear to an experienced Interview Engineer whether code is idiomatic enough.

Examples of non-idiomatic code:

- Using integers 1 and 0 or strings "True" and "False" in a language which supports booleans.
- Using a custom "default" value in a language which supports null or None where appropriate

### Syntax Options:

**Syntax Insufficient data**: The candidate for some reason did not code or wrote not enough code to feel comfortable making a call on their syntax. For example, they decided to leave the interview before the coding section or wrote a trivial solution to the first question with only a few lines of code that was never compiled.

**Pseudocode only / unable**: The candidate demonstrates almost no proficiency in their chosen language. This usually manifests as either:

- A candidate writes just English sentences.
- A candidate writes code which never compiles or runs without help.

**Many errors**: The candidate's code includes many errors. A few "large" errors may also put a candidate in this category. A large error may for example be very incorrect code to perform a simple task such as printing a debug statement or initializing a variable.

**Few errors**: Most candidates fall into this category. The candidate can get work done in their chosen language, but they require help from the compiler output, the Interview Engineer or the web at some points.

**Near perfect**: The candidate demonstrates strong proficiency in their language of choice. They quickly fix minor syntax errors and understand the compiler output with ease. They use idiomatic code.

## Code Style: Style

The Style rating is intended to give a client signal of the quality of the code that a candidate will produce in terms of maintainability. Candidates are encouraged to aim for completeness, and so we do not expect code to be as neat as it might be in a production codebase. That said, strong candidates write code which is neat and easy to follow, as well as code which it is easy to add features to.

High quality production code usually includes comments. We do not expect these in the interview situation due to time constraints.

The style should refer to the final state of the code, for example if the code used poor variable names, but the candidate fixed it, do not select "poor variable names" (though it might be worth commenting that they were poor and then fixed, it gives a useful signal).

## Style Options:

**Style Insufficient data**: The candidate for some reason did not code or wrote not enough code to feel comfortable making a call on their style. For example, they decided to leave the interview before the coding section or the solution did not have enough code to determine if best practices would be followed or if it would end up being easy or hard to maintain.

**Spaghetti code**: Code that flagrantly violates the principles of structured, procedural programming. Extremely difficult, if not impossible, to debug and maintain. The code has a tangled and arbitrary program flow.

**Hard to understand/maintain**: The code shows some characteristics of easy to understand code, but also multiple characteristics of spaghetti code.

**Easy to understand/maintain**: Mostly average or crystal clear. This is typically the default option if no checkboxes are selected.

**Beautifully written**: This should be selected if the candidate wrote code with exceptional high quality. "Beautifully written" code is usually concise and clear.

- All variable names are easy to understand (except for maybe one or two).
- Helper functions are pure and easily testable.
- The approach as described is clearly understandable from reading the code.
- The code style is consistent.

## Style Checkboxes

The available checkboxes give insight into common causes of hard to maintain code in interviews:

- Did not use helper methods when appropriate
- Implementation is more complex than what the approach calls for
- Numerous variable identifiers were confusing, misleading, or impaired readability
  - Used when there are multiple instances of poor variable identifiers being used. This includes highly abbreviated (one or two characters) names, non-descriptive names such as `map`, `set,` or `list`.
  - However, keep in mind that in an interview setting naming typically takes a backseat to problem solving so borderline variable identifiers that are still understandable with context are okay. Likewise, one or two poor names on their own do not constitute this checkbox. The idea here is to call out a candidate that demonstrates a pattern or habit of not following proper naming conventions.

- Used magic numbers
  - Used hardcoded numbers or values without explanation or understanding. For example, subtracting 1 from the returned result because guess and check said so, or using "FOOBAR" or -1 as a special default or stop value rather than None/null/nil.
  - This could also mean hardcoding sizes of a sample input or otherwise writing a solution that would only work for a specific input.

- Inconsistent indentation/spacing
  - This is intended for candidates that demonstrate a pattern of poor whitespace practices. This could mean inconsistent indentation, huge blocks of whitespace, or no whitespace usage at all.
  - It is not intended for one or two mis-indented blocks of code.

- Used global variables
  - If a candidate relied on global variables when it could otherwise be avoided this checkbox is applicable.
  - This is not meant to capture use of member variables of a class in an OOP language.

- Had excessive duplicated or repeated code
  - This is intended for candidates that copy and paste parts of their code rather than using a helper method when it would have been beneficial to do so.
  - This could also encompass not using a temporary variable for repeated deep data structure access like `map[key][array_index]` instead of `value = map[key][array_index]` and then using `value`.

- Had unnecessary deeply nested code
  - This is intended for candidates that use excessively nested logic to the point that it detriments understanding of the code. There is no hard limit to this as it is dependent on language and conventions, but 4+ levels of nesting is when it may become applicable.

## Code Style: Fluency

The fluency score gives an indication of how quickly and comfortably a candidate can write code, given an idea.

In some ways this relates to typing speed, but more often it is an indicator of the candidate's familiarity with their language of choice, programming constructs and ability to translate an abstract idea to code. It is NOT strictly a measure of how quickly a candidate solves one of the programming questions or debugs logic errors in their code. For example, is the candidate able to effortlessly and swiftly write code or do they hesitate and need to think through syntax/code structure frequently?

### Fluency Options:

**Fluency Insufficient data**: The candidate for some reason did not code or wrote not enough code to feel comfortable making a call on their fluency. For example, they decided to leave the interview before the coding section. If they otherwise only wrote a small amount of code and generally performed poorly in the interview then "Limited" is probably the correct option to select.

**Limited**: The candidate has major or constant trouble translating thoughts to written code. They struggle or are unable to produce meaningfully-working code in a reasonable timeframe.

**Proficient**: The candidate is able to write code, but has some trouble efficiently doing so or simply works at a steady pace.

**Native**: The candidate produces code without any problems. They easily and swiftly write code that illustrates their current thought.

### Fluency Checkboxes

- Tested incrementally during implementation
- Tested additional test cases unprompted

## Code Style: Thoughts

Summarize the coding style of the candidate as observed during the collaborative coding exercise.

Guiding Questions:

- Did the candidate have a good grasp of the syntax in the chosen programming language?
- Did the candidate test code in an effective manner?
- At what pace did the candidate work?
- Did the candidate understand the error messages provided by the compiler?
- Was the code easy to read and well organized?

### Writing Samples

**Note:** These samples are for example only. Make sure your write-up is always customized to reflect the specific candidate you have.

#### Code Style Writing Sample 1 - (Poor variable identifiers)

*Ernie worked steadily and made progress at a good pace. His code was generally neatly laid out, though his variable identifiers were highly abbreviated, which reduced its readability.*

#### Code Style Writing Sample 2 - (Many large errors and inconsistent indentation)

*Ralph was fairly comfortable writing JavaScript but still made a lot of minor mistakes. He fixed some easily on his own, but corrected others only with assistance from the interviewer. His code was clear other than some minor inconsistencies in indentation.*

#### Code Style Writing Sample 3 - (No helper methods and more complex than approach calls for)

*Bruno wrote Python with no major errors. However, he continually worked under the erroneous assumption that dictionary values were not mutable, which resulted in some unnecessary extra code. His code was otherwise well structured and easy to understand.*

Additional Resources: **Coding Style: Scoring and Write-up Guide**

## Communication

### Communication: Scoring

Communication is a key part of being a software engineer in a team. Only a small part of a candidate's true communication skills can be assessed in our interview, but we want to give the client the information we have.

Candidates are scored against four communication-related metrics:

- Clarity of Speech
- Quality of Explanation
- Comprehension
- Interaction

### Communication Checkboxes

For each metric, there are optional checkboxes that can be used to capture rarer events. To illustrate, Interaction is a required criteria that can always be reasonably judged -- were the candidate's interactions with you polite and professional, or not? The optional criteria in the Interaction bucket include things like 'Became Frustrated' if the candidate became visibly frustrated, or 'Argumentative/Defensive' for when the candidate is always trying to fight any suggestions of improvement made by the Interview Engineer. Neither of these happen very often, but when they do happen they are very important (in these cases, negative) signals that we want to call out to clients.

## Communication: Clarity of Speech

How well the Interview Engineer understood the words the candidate spoke.

The candidate's speech does not have to be perfect - all that matters is how easily you could understand it. For example, a candidate might have a slight-to-moderate accent but still be perfectly understandable.

Internet connection issues can obviously impact the signal quality, which in turn impacts how clearly a candidate's speech comes across. Interview Engineers should, as best as possible, attempt to grade this criterion independent of the signal quality.

### Clarity of Speech Options:

**Easy to understand**: few or no clarifying questions along the lines of "I didn't understand; could you repeat what you just said?"

**Some challenges understanding**: Some clarifying questions are needed.

**Difficult to understand**: candidate has to be repeatedly asked to reiterate the words they were saying.

### Clarity of Speech Checkboxes

- **Spoke too quickly**: the candidate spoke so quickly that it made them difficult to understand.
- **Mumbled**: the candidate mumbled at various points in the interview.
- **Language proficiency significantly impaired communication**: the candidate's English proficiency made understanding them considerably difficult to the point that would affect day to day communication with them.
- **Poor audio connection impaired communication**: The candidate's computer setup was improper for the interview. Avoid scoring down the "clarity of speech" field for this as it's usually out of the candidate's control if the audio connection negatively affected communication.

## Communication: Quality of Explanation

This dimension deals with how well the candidate explained their thoughts, independent of the

`Clarity of Speech` of the candidate.

Does the candidate choose their words well, structure their ideas in an understandable way, and explain their ideas clearly and concisely?

To illustrate the distinction between Clarity of Speech and Quality of Explanation:

Do you understand the words they are saying well (indicating good `Clarity of Speech`) yet find yourself asking multiple clarifying questions (indicating poor `Quality of Explanation`)?

## Quality of Explanation Options:

**Organized, succinct explanations**: Candidate structures their ideas and explanations clearly and concisely, making them notably easy to understand. They begin by setting appropriate context before diving into details.

**Clear explanations**: Candidate explains concepts clearly and effectively. Explanations may occasionally be unorganized or not fully contextualized, but are still easy to follow.

**Some challenges understanding explanations**: Occasional clarifying questions are needed.

**Difficult to understand explanations**: Struggles to explain ideas. Gets lost in details, or fails to explain context.

### Quality of Explanation Checkboxes

- **Rambled**: The candidate repeatedly took longer than necessary to explain ideas.
- **Digressed**: The candidate repeatedly digressed into irrelevant details.
- Did not ask questions when necessary
- Did not build off of prompts

## Communication: Comprehension

Indicates how well the candidate understood the Interview Engineer.

- Did the candidate understand what was being asked of them?
- Did they easily incorporate the Interview Engineer's feedback?
- Did the Interview Engineer need to repeat themselves multiple times? (NB: This only tracks general, non-technical comprehension. If a candidate understands you but e.g. doesn't know what a technical term like 'priority queue' means, that should not affect this Comprehension score.)

## Comprehension Options:

**Easily understood Interview Engineer**: Candidate clearly understood the Interview Engineer all or almost all of the time, and easily incorporated any feedback.

**Some challenges understanding Interview Engineer**: The Interview Engineer occasionally needed to repeat themselves.

**Difficulty understanding Interview Engineer**: Candidate had significant struggles understanding the Interview Engineer. The Interview Engineer frequently needed to repeat themselves.

### Comprehension Checkboxes

- **Did not answer questions**: The candidate's response to questions showed a fundamental misunderstanding of what the Interview Engineer said or was asking for.
- **Did not adjust to Interview Engineer's feedback**: The candidate did not understand, and therefore failed to properly implement, hints or feedback from the Interview Engineer.

## Communication: Interaction

Indicates whether the candidate interacted with the Interview Engineer in a polite, professional manner. Most interactions with candidates go well, but when they don't, it's an important signal to call out.

### Interaction Options:

**Notable interpersonal skills**: The candidate actively engages with the Interview Engineer and demonstrates enthusiasm for the role and/or the interview itself. Candidate is uniformly polite and friendly, and may proactively ask for clarification or establish parameters of the interview and exercise.

**Smooth interaction**: The candidate behaves appropriately and is polite and professional throughout the interview. Candidate listens to instructions and asks appropriate questions.

**Minor problems with interaction**: Candidate was rarely impolite or unprofessional. Even if it happens once, that should be enough to earn this lower notch.

**Poor interaction**: The candidate was repeatedly impolite or unprofessional. Anything more than once should probably fall under this category, given how rare poor interactions are, and how candidates are generally already trying to be on their best behavior.

### Interaction Checkboxes

- **Argumentative/Defensive**: The candidate was overly defensive or argumentative when their actions were questioned or a mistake was pointed out.
- **Became frustrated**: The candidate became visibly frustrated during the interview.
- **Interrupted Interview Engineer**: The candidate repeatedly interrupted the Interview Engineer.
- **Unenthusiastic / disengaged**: The candidate showed active disinterest in the interview process and/or content.
- Late to interview
  - More than three minutes past the start of the interview.
- Poor location for interview
  - For example, locations with large amount of background noise such as libraries or coffee shops.
- Unprepared for interview
- Used excessive profanity
  - Incidental profanity is generally not a cause for concern. However, if it bothers you then this checkbox is generally applicable.
- Did not follow Interview Engineer's instructions
  - For example, if a candidate continues to work after you have asked them to stop working and started on the interview conclusion.

## Note about profanity

- If someone mutters a swear word once or twice under their breath while debugging code, let it slide.
- If someone swears gratuitously and loudly, or in a clearly negative way towards others, say during the Project Discussion, reduce the Interaction score and explain in Thoughts.

### Communication: Thoughts

Summarize the interaction with the candidate. Justify any negative checkboxes used in this section.

Guiding Questions:

- Was the candidate professional and friendly during the interview?
- What was the quality of the explanations provided?
  - During the project discussion
  - During the coding exercise
- Did the candidate struggle to communicate clearly? Were there any communication issues that should be called out?
- How was the overall interaction?

## Writing Samples

**Note:** These samples are for example only. Make sure your write-up is always customized to reflect the specific candidate you have.

### Writing Sample 1 (Rambled and setup issues)

*Priya was friendly and polite. They had a tendency to ramble for long stretches, and started the interview on a tablet, only retrieving their computer when it became clear that they had to write code that involved a command line shell.*

### Writing Sample 2 (Minor connection issues)

*Frank was friendly and polite throughout the interview. He gave clear explanations of both the project and the algorithms. The interview was interrupted for several minutes due to a network connectivity issue, however, Frank recovered well and returned to the coding exercise largely unfazed.*

### Writing Sample 3 (Good interaction)

*Camille was friendly and easy to engage with throughout the interview. She gave concise, well-structured answers to the knowledge questions and explained her code clearly during the coding exercise.*

### Writing Sample 4 (Good interaction)

*Avery was polite and professional. They provided clear descriptions during the discussions and analysis, coding exercise approaches, and code.*

### Writing Sample 5 (Good interaction)

*Wilma was friendly and engaging throughout the interview and demonstrated a lighthearted but professional sense of humor. She talked through her work in the coding exercise clearly and in depth, and she expressed thoughtful reasons behind her answers in the JavaScript Discussion and Analysis questions.*

### Writing Sample 6 (Good interaction)

*Victoria was friendly and thoughtful. She explained her thoughts with particular clarity. She preferred coding in silence.*

### Writing Sample 7 (Poor interaction)

*Don struggled to communicate ideas clearly. They provided overly verbose and sometimes confusing answers to the knowledge questions. The description of their approach to the coding exercise focused on the code and not on the algorithm. They wrote their solution mostly in silence.*

**Format Variants**

Interview format variants usually take the place of the Project Discussion.

## Discussion and Analysis Questions (DAQs)

Only certain clients request use of the Discussion and Analysis questions. Sometimes Discussion and Analysis questions are used instead of the Project Discussion; sometimes the Project Discussion is shortened and followed by a short set of Discussion and Analysis Questions. Consult the client script for details.

Discussion and Analysis Questions are sets of technical questions with straightforward answers. On the results form, you can indicate correctness by selecting appropriate checkboxes.

### Discussion and Analysis Questions: Thoughts

Please include some information about the candidate's answers to each question in these Discussion and Analysis Questions, regardless of whether they answered correctly or not. The purpose is to give the hiring manager some additional insight into the quality of the answers the candidate provides as this has been requested to assist them in making hiring decisions.

For example, if in Q1 the candidate lists 7+ items, you might phrase it like this, "`<Candidate>` listed a number of useful test cases, including `<test case 1>`, `<test case 2>`, `<test case 3>`." You could also use bullet points or a numbered list to list out the items.

Please also use the Thoughts to:

- Explain that the candidate ran out of time before getting to a particular question (for example, if they only had time for one of the three questions in a section.)
- Explain that the candidate did not answer any of the questions correctly
- Explain why an unusual or borderline answer was marked correct or incorrect

- Mention that you provided hints. You should not provide hints on Discussion and Analysis questions, but just in case it happens.

## Discussion and Analysis Questions: Example 1

### Systems Internals:

*Steve described threads as the basic unit of execution, and processes as collections of threads. For communicating between threads he described using shared memory, and for inter-process communication, he talked about using use mailboxes or queues and shared memory which is not a valid answer in modern operating systems.*

*Steve correctly described how variables are allocated and deallocated on the heap and stack. He explained that stack variables are allocated on the current stack frame and are fixed in size, and heap variables are allocated and deallocated using `malloc` and `free`.*

*Steve described traditional memory management as requiring the developer to clean up after themselves or use statically defined variable space, as compared to garbage-collected languages that handle memory allocation and cleanup. He talked about real-time applications being inappropriate for garbage collection due to memory fragmentation issues and timing issues.*

### Object Oriented Design:

*Steve described inheritance as one class deriving from another and extending its functionality, and composition as embedding objects in other objects. He would use composition for resource access such as network connections or sockets, where the class using the connection is not an extension of the connection itself.*

*Steve did not know the term dependency injection but gave a reasonable justification for its use when the term was explained to him.*
*For a new collection type, Steve would document any scenarios that would throw an exception, any restrictions on the types that could be stored, and the thread safety of the collection.*

## Discussion and Analysis Questions: Example 2

### Production Issues:

*Steve identified the problem as a memory leak and said he would inspect the source code to try find it, which is not a sufficient answer.*

*Steve did not describe any symptoms of thread exhaustion. He said he would use a thread scheduler or locking structures to alleviate the problem, neither of which are valid answers.*

*Steve talked about load testing the web service by sending 1000 requests at it and seeing if it could respond in under a second.*

### Testing:

*Steve did not know what mocks were and chose to skip this question.*

*Steve said he would determine which tests are slow using timing logs.*

*For testing the graph algorithm, Steve gave the following cases:*

- *Two nodes with a valid route between them*
- *Two nodes with multiple routes between them, ensuring the shortest is picked*

## Discussion and Analysis Questions: Example 3

This is an example of an interview with iOS Discussion and Analysis questions where the candidate answered all the questions and very little needed to be written.

### Question 1:

*Steve correctly defined the Singleton pattern and talked about the potential for memory leaks and problems with global state as reasons not to use it.*

### Question 2:

*Steve talked about many different ways of handling updates, in descending order of preference.*

*First, he described using the Notification system, then closures or delegates. He talked about using a reactive approach using observers, but stated that wasn't as clean as notifications. He finally touched on polling, and said that was only appropriate in very limited circumstances.*

### Question 3:

*Steve talked about using a cloud storage system like Firebase which wasn't a valid answer, but went on to talk about CoreData, SQL databases, and NSUserDefaults, with a preference for CoreData as it provides a full object graph which he found easy to work with.*

### Question 4:

*Steve explained reference counting in detail and described the difference between strong and weak references clearly.*

## Discussion and Analysis Questions: Example 4

### Question 1:

*Steve identified that the given approach does not meet the requirements because longest lines will be the most commonly hit due to taking up more space inside the file.*

### Question 2:

*Steve correctly identified the time complexity of the method as O(log x).*

### Question 3:

*Steve correctly identified the time complexity of `is_contained` as `O(n + m)` and `is_contained_2` as `O(n * log(m))`. He misidentified the second function as having better asymptotic time complexity and ran out of time*

*before answering the last follow-up question.*

## Offline Coding Challenge (Offline Code)

### Offline Code: Scoring

In the results form, you'll be asked to grade their level of understanding of their implementation, as well as whether they were able to identify the time and space complexities of their implementation.

In some cases, their explanation will be so poor that you aren't able to discern whether their time and space complexities analysis was correct. In this case, explain your uncertainty in the Thoughts section, and err on the side of marking it as correct.

### Offline Code Level of Understanding Options:

Note that these are not prescriptive; some candidates may show varying levels of understanding of their approach and code, so use your best judgement to determine which option best fits their performance.

Minimal: The candidate struggled to explain their approach or what their code did. They did not have much if any insight into other ways to approach the problem.

Moderate: The candidate gave a brief overview of their approach, and explained their code line by line. They may have another approach in mind.

Above-average: The candidate gave a good description of their approach, and explained their code in pieces rather than just reading the code. If there was another approach they could have taken, they explain it well and describe the trade-offs. Otherwise, they explain why their solution is optimal. They may share some insight into the structure of their code.

Significant: The candidate succinctly explains their approach and how their code solves the problem. They explain how they structured their code clearly, as well as any trade-offs in their approach or code structure.

Not enough info: The candidate either did not answer the question or their explanations were completely unintelligible.

### Offline Code: Thoughts

For the written part of the results form, you don't need to go into too much depth. Describe in a few sentences what they said to you at a high level, how well they explained it, and how well they appeared to understand what they were talking about.

### Offline Code Writing Sample 1

*Jen demonstrated a strong understanding of the offline problem, effectively summarizing her approaches and correctly identifying the time and space complexities.*

### Offline Code Writing Sample 2

*Carla's walkthrough of her offline coding submission was clear, but she struggled some to remember her solution. Her code included inefficient data structure choices and was not optimal. She recognized the exponential time complexity but was not confident in her analysis.*

## Object Oriented Design Code Review

### Object Oriented Design Code Review: Scoring

Each issue is intentionally isolated so that it can be pointed out individually. As a result, we can classify individual issues on a spectrum of difficulty/obviousness and assign corresponding points.

Any additional comments will also be noted in the additional thoughts section, e.g. "More comments explaining the models would be good to have."

| Issue | Issue Identified | Issue Fixed |
|---|---|---|
| Controller initializes new connection to the DB | 2 | 2 |
| "String constant like ""root"" or ""shapes"" should not be hardcoded string literals" | 1 | 1 |
| Mapping of Shape.type to String (via switch) in controller | 2 | 1 |
| `translate()` contains business logic | 3 | 1 |
| Poor SQL practices in `translate()` | 1 | 2 |
| Input parameters are not validated | 1 | 1 |
| Shape.type is a public and settable int | 1 | 1 |
| getArea() should be abstract | 1 | 1 |
| Circle.setWidth() and setHeight() violate the Liskov substitution principle | 3 | 2 |
| Integer division loses precision | 2 | 1 |
| squareNum() is unnecessarily public | 1 | 1 |
| Point is not a shape with area | 2 | 3 |

### Bucketing

- Beginner: [0, 10)
- Intermediate: [10, 20)
- Proficient: [20, 30)

- Expert: [30, 38]

### Object Oriented Design Code Review: Thoughts

Any additional issues that were not listed in the previous section can be pointed out in the Thoughts field, as well as any observations of how the candidate approached the problem in general. Strong candidates tend to take a pretty methodical approach, which can be noted. Weaker candidates might spend little to no time overall on the code review, they might look for issues that are irrelevant, etc. All of these can be called out in the writeup.

## Written Feedback to the Candidate

**WARNING**

**The written feedback should be brief and in bullet-point format.**

Please provide a few bullet points of feedback. This feedback is not sent directly to candidate, but passed on by recruiters when candidates reach out for feedback. It is recommended to leave 1-2 things that were done well and 1-2 areas/suggestions for improvement.

**Do not reveal any information about how the candidate performed in relation to other candidates, or any information about whether you think they "passed" or "failed" or that they did great or not overall. Remember, we don't make decisions about candidates.**

The written feedback should be a brief summary of *what* happened during the interview. Be cautious about passing judgment on *how* they worked, lest it come across as dismissive of their style or past experiences. Unless a candidate explicitly asks for advice on *how* they can improve, it's best to give them observational information and let them figure out what to do with it.

### Example Items to Provide as Feedback

You may find the following list helpful. It is compiled and summarized from written feedback that Interview Engineers have previously given internal candidates:

#### Introduction Example

- Be prepared to give a one minute elevator pitch about yourself and your technical background.

#### Project Discussion Example

- Be prepared to talk about a technical project.
- Focus on a technical challenge and how you handled it.
- Non-technical aspects aren't as important in an interview setting.
- Ask yourself, "What skills do I want to highlight to a potential employer?"

#### Coding Exercise Example

- Practice, practice, practice!
- Think through the algorithm and think about code design ahead of time.

- Start by writing the simplest parts, like the method signature(s), to get comfortable.
- Identify straightforward solutions first, and then think about how to improve it.
  - Think about this before writing code, since the interview will move on once you have working code.
  - Completely working code is better than partially working code, regardless of approach.

- Always verbalize your approach, especially key decisions and reasoning.
  - Your Interview Engineer can read code and will ask questions as necessary.

- Discuss trade-offs of potential solutions: your Interview Engineer will steer you toward what they want to see in a solution.
- Ask clarifying questions, the Interview Engineer might be intentionally vague to see if you make assumptions.
- Break the problem down, break the code into smaller functions.
- Choose a language best suited for the problem at hand; choose a language where you can shine!
- Use the runtime environment and interpreter, if relevant to your language, to your advantage.
- Brush up on big-O and be able to analyze time and space complexity of common data structure operations and code flow.
- Don't forget about edge cases. Propose additional test cases to check all boundary conditions and increase confidence in the correctness of your code.
- Brush up on standard libraries of languages you're familiar with, you will mostly likely need to use data structures in your solution.
- Keeping a dialogue going with the Interview Engineer while coding can help the Interview Engineer better follow along and guide you, if necessary.

## Mock/Practice Interview Feedback Process

Check out the mock/practice Handbook section starting at FEEDBACK DIALOGUE.

### Feedback Writing Sample 1

- *Ensure that you give yourself the opportunity to fully develop your approach before you begin implementing your code. Try to look at each piece of information and the sample input thoroughly before you begin. Ask for some silent time with your thoughts if the Interview Engineer seems impatient, but try to keep them up-to-date with your thoughts as they develop.*
- *We didn't discuss this but be sure to brush up on Big O / time & space complexities so that you can describe the efficiency of your approach.*
- *Don't be afraid to get deeper into a nuts and bolts discussion of your project when that comes up. Talk about the code and how you organized it.*

If no verbal feedback was provided during the interview:

Powered by Zendesk

- *Remember to give yourself the luxury of working in silence and comfort to develop your approach. It is crucial that you formulate a path towards a solution that you can implement. Strive to know exactly how you will implement your code before you begin.*
- *When you decide to remove a portion of code from your solution, do it entirely. Otherwise, it will have the effect of confusing the mind if we leave it only commented out.*

- *Try to practice with a friend to replicate the setting of the interview like we did together. Repeated practice will bring excellent improvement, just stick at it!*

### Feedback Writing Sample 3

For interview formats where the feedback is the point of the interview. Make sure to have a mix of positive and negative feedback!

*Introduction*

- *Always be prepared to give a 1 minute elevator pitch about yourself and your technical background. You did a good job stating where you are at, but make sure to provide more relevant details about your experience and technologies you worked on.*

*Project discussion*

- *Be prepared to talk about a project. This kind of question comes up often in interviews, so have one at the ready that you can comfortably talk about in depth.*
- *Focus on a technical challenge and how you handled it.*
- *Make sure that you connect all the technical aspects of the project that you are working on and to drill down on some details. You talked about a few different SQL queries that you wrote which is great, but you'll show a better understanding of the project if you can link it in to some larger project goal (say, improving page performance) and explain how that helped achieve the goal.*
- *Ask yourself what skills do you want to highlight to a potential employer?*

*Coding*

- *Practice, practice, practice! Online coding environments can be a bit different to what you're used to, so if they provide the option of trying it out before the interview starts, have a quick go at using it so you're not surprised during the interview.*
- *You did a good job of explaining your assumptions during the approach discussion. Don't be afraid to ask clarifying questions, as the Interview Engineer might be intentionally vague to see if you make assumptions.*
- *Think through the algorithm and think about how you'll structure your solution before starting to code. Quickly listing out the steps in comments can help, so the coding process just becomes turning those comments into code.*
- *Always verbalize your approach, especially key decisions and reasoning.*
- *Brush up on big-O and be able to analyze time and space complexity of common data structure operations and code flow.*
- *Brush up on standard libraries of languages you're familiar with, you will mostly likely need to use data structures in your solution.*
- *You did a great job of cleaning up your code as you went, but check with the Interview Engineer to see code quality is scored. You don't want to spend time on something that isn't being evaluated!*

## Candidate Decides to Terminate Interview Early

If the candidate opts to end the interview early, we should mention this in the Thoughts paragraph in the section in which they decided to terminate the interview.

Be careful to keep the tone neutral and do not speculate as to why they decided to end the interview early, just state the fact that they did.

### Example - Candidate terminates interview during the first coding question

#### Q1 Thoughts

Jill proposed a solution to iterate through the input to identify the first zero and last zero as the top-left and bottom-right coordinates respectively. She struggled to translate her approach into code. She then opted to end the interview twenty minutes early.

## When would I *not* need to submit a write-up?

If the candidate chooses to end the interview early, a write-up is most likely still needed. If the interview ends early due to issues that are out of the candidate's control, such as technical problems, then we will reschedule the interview and you may not need to submit a write-up.

## Additional Resources

- Coding Style: Scoring and Write-up Guide
- Real World Coding - Exercises with existing codebases (excluding NextGen)

**Previous article**

Ending the Interview

**Next article**

Coding Style: Scoring and Write-up Guide